DiscRec: Disentangled Semantic–Collaborative Modeling for Generative Recommendation

Chang Liu*, Yimeng Bai†, Xiaoyan Zhao‡, Yang Zhang§, Fuli Feng†, and Wenge Rong*
*Beihang University, Beijing, China
†University of Science and Technology of China, Hefei, China
‡The Chinese University of Hong Kong, Hong Kong, China
§National University of Singapore, Singapore, Singapore
chang.liu@buaa.edu.cn, baiyimeng@mail.ustc.edu.cn, xzhao@se.cuhk.edu.hk,
zyang1580@gmail.com, fulifeng93@gmail.com, w.rong@buaa.edu.cn

***Abstract*— Generative recommendation is emerging as a powerful paradigm that directly generates item predictions, moving beyond traditional matching-based approaches. However, current methods face two key challenges: token-item misalignment, where uniform token-level modeling ignores item-level granularity that is critical for collaborative signal learning, and semantic–collaborative signal entanglement, where collaborative and semantic signals exhibit distinct distributions yet are fused in a unified embedding space, leading to conflicting optimization objectives that limit the recommendation performance.**

To address these issues, we propose DiscRec, a novel framework that enables Disentangled Semantic-Collaborative signal modeling with flexible fusion for generative Recommendation. First, DiscRec introduces item-level position embeddings, assigned based on indices within each semantic ID, enabling explicit modeling of item structure in input token sequences. Second, DiscRec employs a dual-branch module to disentangle the two signals at the embedding layer: a semantic branch encodes semantic signals using original token embeddings, while a collaborative branch applies localized attention restricted to tokens within the same item to effectively capture collaborative signals. A gating mechanism subsequently fuses both branches while preserving the model's ability to model sequential dependencies. Extensive experiments on four real-world datasets demonstrate that DiscRec effectively decouples these signals and consistently outperforms state-of-the-art baselines. Our codes are available on https://github.com/Ten-Mao/DiscRec.

***Index Terms*—generative recommendation; collaborative signal; disentangled representation**

I. Introduction

Generative recommendation, emerging as a promising next-generation paradigm, has attracted increasing attention from both industry and academia [1]–[6], as it departs from the traditional discriminative query-candidate matching framework by directly generating the next item in an autoregressive manner. Typically, this paradigm involves two stages: (1) a *tokenization stage*, where a tokenizer model employs discretization methods (*e.g.,* RQ-VAE [7]) to convert the specific representation of items into corresponding semantic IDs [8]–[12]; (2) a *recommendation stage*, where a Transformer-based recommender model (*e.g.,* T5 [13]) autoregressively generates item predictions based on the semantic ID sequences constructed from user interaction histories.

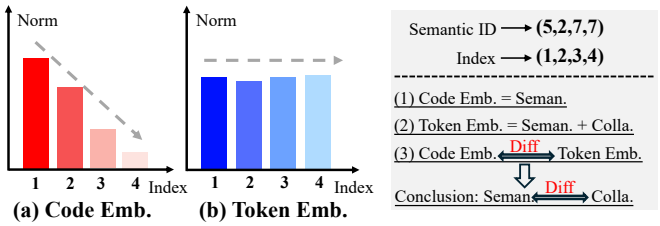

Fig. 1: Comparison of the distribution between code embeddings (*i.e.,* entries from the RQ-VAE [7] tokenizer's codebook) and token embeddings (*i.e.,* entries from the T5 [13] recommender embedding table) across indices of the semantic ID, using embedding norm as a measure of information capacity and averaged over all items.(a) Code embeddings show a clear hierarchical decay, while (b) token embeddings have a more uniform distribution. The right side illustrates how code embeddings primarily encode semantic signals, while token embeddings incorporate both semantic and collaborative signals. This difference reflects the distinct distributions of these two signal types.

Within this paradigm, the recommender undertakes two core tasks: extracting *semantic signals* embedded in the semantic IDs, and capturing *collaborative signals* through the modeling of sequential transition patterns in user interaction histories. Therefore, superior generative recommendations fundamentally relies on the seamless integration of these two heterogeneous information sources. From this perspective, we identify two key challenges: (1) **Token-item misalignment.** Existing generative recommender models [8], [9], [14] treat all tokens uniformly, ignoring the item boundaries they belong to. This leads to the token-item misalignment, where the lack of item-level granularity undermines the model's ability to effectively learn collaborative signals. (2) **Semantic–collaborative signal entanglement.** These models [8], [9] often entangle collaborative and semantic signals within a unified token embedding space. However, as shown in Figure 1, our empirical analysis reveals that these two signals follow distinct distribution patterns across indices of the semantic ID—semantic signals

exhibit a hierarchical decay, whereas collaborative signals are not. This discrepancy introduces imcompatible optimization objectives, which not only leads to representational interference during training, but also degrades the model's ability to effectively capture either signal—ultimately limiting recommendation performance.

Existing works on generative recommendation primarily focus on leveraging both collaborative and semantic signals but do not directly address the underlying challenges. The first direction focuses on the tokenization stage, where semantic IDs are enhanced through the incorporation of auxiliary collaborative regularization losses [9], [10], [15], [16]. However, it relies on pretrained collaborative embeddings and remains constrained by the downstream recommender's misaligned token-level architecture. The second direction operates at the recommendation stage, introducing a two-stream architecture that builds entirely separate models for semantic and collaborative signals [14], [17]–[19]. While this achieves complete disentanglement at the embedding level, it requires maintaining two isolated token spaces and model flows, which substantially increases computational cost. Moreover, the two streams interact only at the final reranking stage, preventing deep integration and limiting the exploration of their complementary strengths.

Building upon the limitations of prior approaches [9], [14], we distill our target for generative recommender models as achieving **item-aware alignment** and **semantic-collaborative signal disentanglement with flexible fusion**. Initially, we aim to incorporate item awareness to align the modeling granularity with the item-level nature of recommendation tasks. Based on this foundation, we further introduce a flexible disentanglement mechanism that separates semantic and collaborative signals explicitly at the token embedding layer to mitigate representational interference during optimization. In contrast to two-stream architectures enforcing strict separation throughout the model, we retain cross-signal interactions at deeper layers, allowing the model to fully leverage their complementary strengths for improved recommendation performance.

To this end, we propose DiscRec, a novel framework that enables Disentangled Semantic-Collaborative signal modeling with flexible fusion for generative Recommendation. First, to achieve item-aware alignment, we introduce item-level position embeddings for each token, with positions assigned according to their indices within the corresponding semantic ID sequence. By sharing these embeddings across different items, we enable the model to efficiently discern the item-level structure within the input token sequences. Second, to enable semantic-collaborative signal disentanglement with flexible fusion, we propose a dual-branch module within a single workflow. The semantic branch directly models semantic signals using the original token embeddings. In parallel, the collaborative branch combines token embeddings with item-level position embeddings and employs a Transformer with localized attention, where attention is restricted to tokens within the same item, to effectively capture collaborative signals. The outputs from both branches are adaptively fused via a gating mechanism, allowing the disentanglement to be seamlessly integrated in a plug-and-play manner at the embedding layer, while preserving the original architecture's ability to model complex sequential dependencies. To demonstrate the generalizability of DiscRec, we incorporate it into two representative generative recommendation frameworks, TIGER [8] and LETTER [9], and systematically validate it through experiments on four real-world datasets. Experimental results demonstrate that DiscRec successfully decouples collaborative and semantic signals and consistently achieves superior recommendation performance.

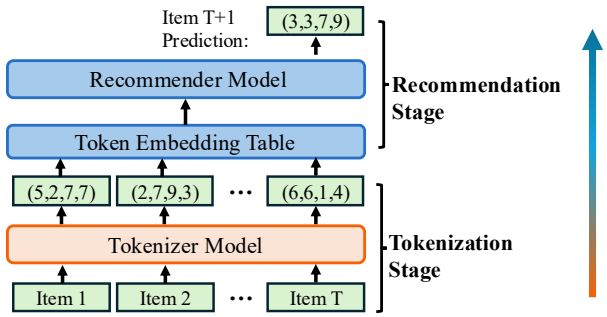

Fig. 2: Overview of the generative recommendation paradigm, which sequentially applies the tokenization stage to discrete the semantic embeddings, and the recommendation stage to generate item predictions. Each tuple (*e.g.,* (5,2,7,7)) denotes a sequence of tokens representing a semantic ID of an item.

The main contributions are summarized as follows:

- We empirically demonstrate that collaborative and semantic signals exhibit significantly different distributions, and we identify two critical challenges in generative recommendation: token-item misalignment, semantic–collaborative signal entanglement.
- We propose DiscRec, a novel framework that integrates item-level position embeddings with a dual-branch module to achieve item-aware alignment and semantic-collaborative signal disentanglement with flexible fusion.
- We conduct extensive experiments on multiple real-world datasets, validating the effectiveness of DiscRec in enhancing generative recommendation performance.

II. Preliminary

In this section, we formally define the task of generative recommendation. Consistent with prior studies [8], [9], we focus on the sequential recommendation setting. Given a universal item set $\mathcal{I}$ and a user's historical interaction sequence $S = [i_1, i_2, \ldots, i_T]$, the objective is to predict the next item $i_{T+1} \in \mathcal{I}$ that the user is likely to interact with. Departing from the traditional query-candidate matching paradigm [20], generative recommendation reframes the task as directly generating the next item. As shown in Figure 2, this paradigm typically consists of two core components:

A. Tokenization Stage

1) Task Formulation: At this stage, each item is encoded into a sequence of discrete tokens by feeding its semantic embedding into a tokenizer model $\mathcal{T}$. Following TIGER [8], a representative generative recommendation framework, we adopt the Residual Vector Quantized Variational Autoencoder (RQ-VAE) [7] to discretize item representations into semantic IDs. This hierarchical encoding offers two primary advantages. First, it inherently induces a tree-structured item space, which is well-suited for generative modeling. Second, items sharing common prefix tokens naturally capture collaborative semantics, enabling effective information sharing among semantically related items. Generally, given an item i, the tokenizer $\mathcal{T}$ takes its semantic embedding $\boldsymbol{z}$—typically a pretrained text embedding and produces a sequence of quantized tokens across L hierarchical levels, formulated as:

$$[c_1, \ldots, c_L] = \mathcal{T}(\boldsymbol{z}), \tag{1}$$

where c_l represents the token assigned to item i at the l-th level of the hierarchy. Then, each level uses a separate codebook to incrementally refine the representation by encoding residuals from the previous level.

2) Model Optimization: We now describe the process of obtaining hierarchical tokens for a specific item i. We begin by encoding the input semantic embedding $\boldsymbol{z}$ into a latent representation using a MLP-based encoder:

$$\boldsymbol{r} = \text{Encoder}_{\mathcal{T}}(\boldsymbol{z}). \tag{2}$$

The latent representation $\boldsymbol{r}$ is then quantized into L discrete tokens by querying L distinct codebooks. At each level l, we have a codebook $\boldsymbol{\mathcal{C}}_l = \{\boldsymbol{e}_l^k\}_{k=1}^K$, where each e_l^k is a learnable embedding vector and K is the codebook size. The codebook acts as a discrete vocabulary that maps continuous latent vectors into symbolic tokens by selecting the nearest code in Euclidean space. Specifically, this quantization process is performed as follows:

$$c_l = \arg\min_k \|\boldsymbol{v}_l - \boldsymbol{e}_l^k\|^2, \tag{3}$$

$$\boldsymbol{v}_l = \boldsymbol{v}_{l-1} - \boldsymbol{e}_{l-1}^{c_{l-1}}. \tag{4}$$

Here, $\boldsymbol{v}_l$ denotes the residual vector at the l-th level ($\boldsymbol{v}_1 = \boldsymbol{r}$), and the assignment is based on the Euclidean distance between $\boldsymbol{v}_l$ and each entry in the corresponding codebook. After quantizing the initial semantic embedding into a hierarchy of tokens from coarse to fine granularity, we obtain the quantized representation $\tilde{\boldsymbol{r}}$, which is then fed into an MLP-based decoder to reconstruct the semantic embedding:

$$\tilde{\boldsymbol{r}} = \sum_{l=1}^{L} \boldsymbol{e}_l^{c_l}, \tag{5}$$

$$\tilde{\boldsymbol{z}} = \text{Decoder}_{\mathcal{T}}(\tilde{\boldsymbol{r}}). \tag{6}$$

Overall, the tokenization loss for item i is defined as[1]:

$$\mathcal{L}_{\mathcal{T}} = \|\tilde{\boldsymbol{z}} - \boldsymbol{z}\|^2 + \sum_{l=1}^{L} (\|\text{sg}[\boldsymbol{v}_l] - \boldsymbol{e}_l^{c_l}\|^2 + \beta \|\boldsymbol{v}_l - \text{sg}[\boldsymbol{e}_l^{c_l}]\|^2). \tag{7}$$

The first term represents a reconstruction loss, which encourages the reconstructed embedding $\tilde{\boldsymbol{z}}$ to closely approximate the original input $\boldsymbol{z}$. The remaining two terms correspond to quantization losses, aiming to minimize the discrepancy between the residual vectors and their associated codebook entries. Here, sg$[\cdot]$ denotes the stop-gradient operation [7], and β is a weighting coefficient that balances the learning dynamics between the encoder and the codebooks.

By apply the discretization process for each item in user history, the input interaction sequence S can be transformed into a token sequence:

$$X = [c_1^1, c_2^1, \ldots, c_L^1, c_1^T, \ldots, c_L^T], \tag{8}$$

where c_l^j denotes the l-th token of item i_j ($j \in \{1, \ldots, T\}$), T is the length of the original interaction sequence S, and L is the length of each item identifier. The ground-truth next item i_{T+1} is similarly represented as a token sequence:

$$Y = [c_1^{T+1}, \ldots, c_L^{T+1}]. \tag{9}$$

B. Recommendation Stage

1) Task Formulation: At this stage, the next-item prediction task is framed as a sequence generation problem over the tokenized interaction history. Specifically, each token of the target item is generated autoregressively, conditioned on the input token sequence and all previously generated tokens. This approach enables the extraction of semantic signals from the semantic IDs produced during tokenization, while capturing collaborative signals through modeling the sequential transition patterns in user interactions.

2) Model Optimization: Consistent with prior work [8], [9], our recommender model $\mathcal{R}$ employs an encoder-decoder architecture based on T5 [13]. For the tokenized sequence X, we append a end-of-sequence token [EOS] and then construct its embedding sequence $\boldsymbol{E}^X \in \mathbb{R}^{(LT+1)\times D}$ by looking up the token embedding table $\boldsymbol{\mathcal{O}}$, where D denotes the embedding dimension. The embedding sequence of X is defined as:

$$\boldsymbol{E}^X = [\boldsymbol{o}_1^1, \boldsymbol{o}_2^1, \ldots, \boldsymbol{o}_L^T, \boldsymbol{o}^{\text{EOS}}], \tag{10}$$

where $\boldsymbol{o}_l^t$ denotes the embedding of the l-th token for item i_t (*i.e.,* token c_l^t). This embedding sequence is then fed into the Transformer encoder to obtain the corresponding hidden representation:

$$\boldsymbol{H}^{\text{Enc}} = \text{Encoder}_{\mathcal{R}}(\boldsymbol{E}^X). \tag{11}$$

For decoding, a special beginning-of-sequence token [BOS] is prepended to the target token sequence Y, resulting in an embedding sequence $\boldsymbol{E}^Y \in \mathbb{R}^{(1+L)\times D}$ defined as:

$$\boldsymbol{E}^Y = [\boldsymbol{o}^{\text{BOS}}, \boldsymbol{o}_1^{T+1}, \ldots, \boldsymbol{o}_L^{T+1}]. \tag{12}$$

[1]Note that additional auxiliary losses, such as collaborative regularization, may also be included (see LETTER [9]).

The Transformer decoder then takes the encoder output $\boldsymbol{H}^{\text{Enc}}$ and the target embedding sequence $\boldsymbol{E}^{Y}$ as input to model the user preference representation:

$$\boldsymbol{H}^{\text{Dec}} = \text{Decoder}_{\mathcal{R}}(\boldsymbol{H}^{\text{Enc}}, \boldsymbol{E}^{Y}). \tag{13}$$

The decoder output $\boldsymbol{H}^{\text{Dec}}$ is projected onto the token embedding space via an inner product with the token embedding table $\mathcal{O}$ to predict the target tokens. The recommendation loss is defined as the negative log-likelihood of the target tokens under the sequence-to-sequence learning framework:

$$\mathcal{L}_{\mathcal{R}} = -\sum_{l=1}^{L} \log P(Y_l | X, Y_{<l}), \tag{14}$$

where Y_l denotes the l-th token in the target sequence, and $Y_{<l}$ represents all preceding tokens.

C. Distribution Analysis of Semantic and Collaborative Signals

To further elucidate our core motivation, we offer a more detailed explanation of the experiment shown in Figure 1, with the aim of demonstrating the necessity of disentangling semantic and collaborative signals. Building on prior work [21], [22], we hypothesize that the norm of an embedding vector serves as a proxy for the information capacity it encodes. Under this assumption, we examine the norm distributions of embedding vectors from both the tokenizer model (RQ-VAE [7]) and the recommender model (T5 [13]) within a representative generative recommendation framework (TIGER [8]).

Specifically, for an item i with semantic ID $[c_1, \ldots, c_L]$, we perform lookup operations on the codebooks $\{\boldsymbol{\mathcal{C}}_l\}_{l=1}^{L}$ and the token embedding table $\boldsymbol{\mathcal{O}}$ to obtain the code embedding sequence and token embedding sequence, each of length L. For each embedding sequence, we compute the norm of each embedding vector. This process is repeated for all items, and the average norm value is then calculated across the entire set. Ultimately, we obtain two vectors of length L, which are used to plot the histograms in Figure 1 ($L = 4$), serving as measures of information capacity across the indices of the semantic IDs.

We observe that code embeddings demonstrate a pronounced hierarchical decay, unlike the more uniform distribution of token embeddings. Given that code embeddings mainly represent semantic information whereas token embeddings integrate both semantic and collaborative information, this difference highlights the fundamentally distinct characteristics of these two signal types. This discrepancy gives rise to conflicting optimization objectives, as semantic signals tend to induce a hierarchical decay in information capacity across the indices of semantic IDs, whereas collaborative signals do not. The associated risk—that the model's inability to effectively represent either signal ultimately impairs recommendation performance—motivates us to disentangle semantic and collaborative signals.

III. Methodology

In this section, we first provide a detailed introduction to the proposed Disentangled Collaborative-Semantic Modeling (DiscRec) framework. As shown in Figure 3, our DiscRec framework consists of two key modules: the *Item-level Position Embedding*, which incorporates item-aware information into the input representations, and the *Dual-Branch Module*, which disentangles and adaptively fuses semantic and collaborative signals. We first describe each component in detail, followed by a complexity analysis.

A. Disentangled Collaborative-Semantic Modeling

DiscRec is designed to address two key challenges in token-level generative recommendation: item-token misalignment and semantic–collaborative signal entanglement. To this end, we introduces two complementary modules. The *Item-level Position Embedding* encodes structural information about each item token's position, enabling the model to distinguish tokens across items. The *Dual-Branch Module* separately models semantic and collaborative signals via parallel encoding and later fuses them adaptively at the embedding level. The following sections provide a detailed explanation of each component.

1) Item-level Position Embedding: The key to incorporating item awareness into the architecture is enabling the model to distinguish tokens from different items rather than treating them equivalently. A straightforward solution would be to assign a unique embedding to each item; however, this approach lacks scalability, as will be discussed in the experimental section. Therefore, we adopt a lightweight alternative by constructing a dedicated *Item-level Position Embedding (IPE)* table $\mathcal{V} \in \mathbb{R}^{(L+2)\times D}$. Specifically, the first L rows correspond to the L tokens of each item, while the last two rows respectively correspond to the special tokens [EOS] and [BOS]. For the token sequence X and Y, the corresponding position embedding sequence is respectively defined as follows:

$$\boldsymbol{V}^{X} = [\boldsymbol{v}_1, \boldsymbol{v}_2, \ldots, \boldsymbol{v}_{L-1}, \boldsymbol{v}_L, \boldsymbol{v}_{L+1}], \tag{15}$$

$$\boldsymbol{V}^{Y} = [\boldsymbol{v}_{L+2}, \boldsymbol{v}_1, \ldots, \boldsymbol{v}_L], \tag{16}$$

where $\boldsymbol{v}_l$ represents the l-th row of the embedding table $\mathcal{V}$. Notably, this embedding assignment is explicitly determined by the token's position within the corresponding semantic ID and is independent of the item itself. By sharing these embeddings across different items, the model can efficiently capture the item-level structural information embedded within the input token sequences.

2) Dual-Branch Module: Overall, this module aims to leverage both the original token embeddings (Equations (10) and (12)) and the introduced position embeddings (Equations (15) and (16)) to reconstruct the inputs to the encoder and decoder (Equations (11) and (13)), thereby achieving the explicit disentanglement of the two types of signals. We denote the whole module as $\mathcal{M}$, and the resulting disentangled hidden representations of encoder and decoder are computed as follows:

$$\boldsymbol{H}_{\text{Dis}}^{\text{Enc}} = \text{Encoder}_{\mathcal{R}}(\mathcal{M}(\boldsymbol{E}^{X}, \boldsymbol{V}^{X})), \tag{17}$$

$$\boldsymbol{H}_{\text{Dis}}^{\text{Dec}} = \text{Decoder}_{\mathcal{R}}(\boldsymbol{H}_{\text{Dis}}^{\text{Enc}}, \mathcal{M}(\boldsymbol{E}^{Y}, \boldsymbol{V}^{Y})). \tag{18}$$

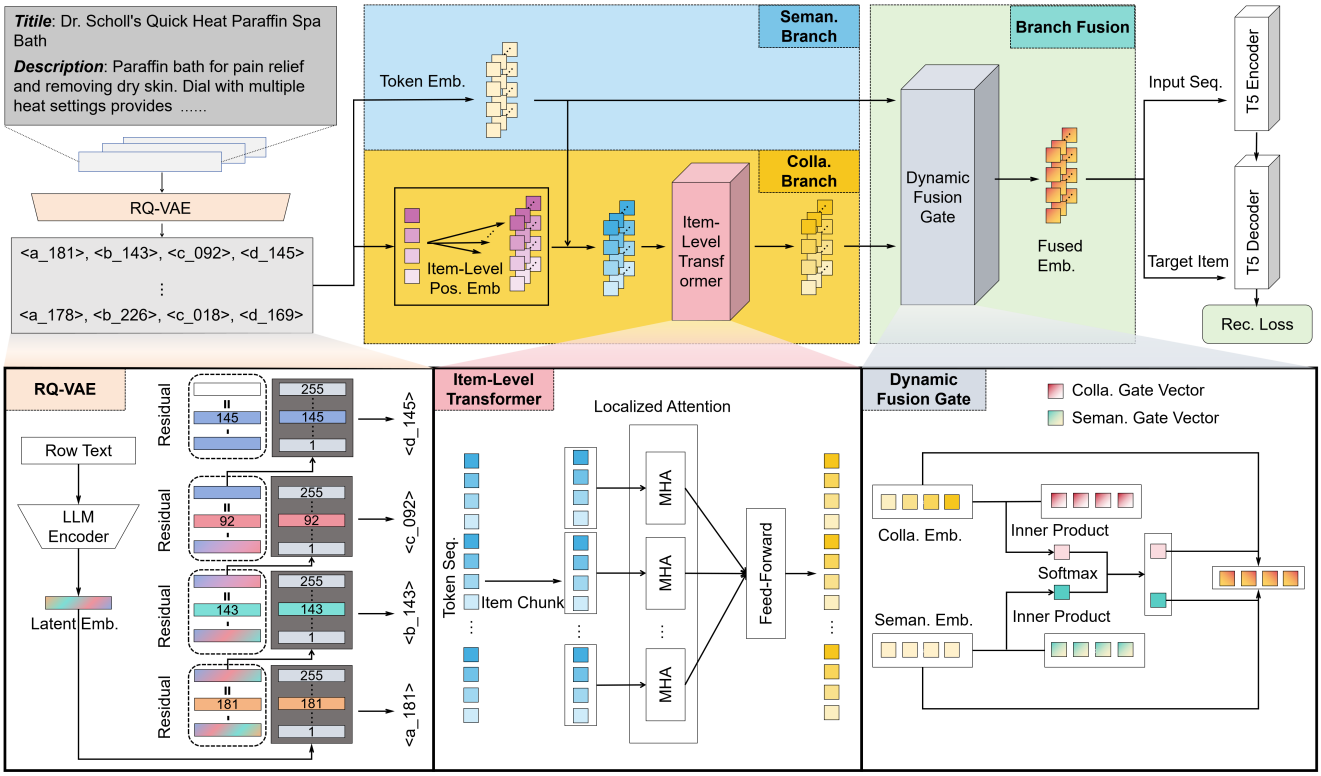

Fig. 3: Overview of the proposed DiscRec framework. The upper part illustrates the generative recommendation pipeline, which sequentially employs tokenization and recommendation. Specifically, we adopt a dual-branch module for recommendation: a collaborative branch (centered on the item-level position embedding and the item-level Transformer) and a semantic branch. These two branches are adaptively fused via a dynamic fusion gate. The three subfigures below provide detailed structures of the RQ-VAE, the item-level Transformer, and the dynamic fusion gate, respectively.

Below, we present the formulation of $\mathcal{M}(\boldsymbol{E}^X, \boldsymbol{V}^X)$; the computation of $\mathcal{M}(\boldsymbol{E}^Y, \boldsymbol{V}^Y)$ can be derived in a similar manner. The dual-branch module $\mathcal{M}$ consists of three components: a semantic branch, a collaborative branch, and an adaptive fusion mechanism. Each component is introduced in detail below:

Semantic Branch. This branch is dedicated to extracting semantic signals from the semantic IDs generated during the tokenization process. Since it is independent of collaborative information, it simply outputs the original token embeddings $\boldsymbol{E}^X$ without any additional processing:

$$\boldsymbol{B}^{\text{Seman}} = \boldsymbol{E}^X \in \mathbb{R}^{(LT+1)\times D}, \tag{19}$$

where $\boldsymbol{B}^{\text{Seman}}$ represents the output of the semantic branch.

Collaborative Branch. This branch aims to capture collaborative signals by modeling the sequential transition patterns inherent in user interactions. To achieve this, both the original token embeddings $\boldsymbol{E}^X$ and the introduced position embeddings $\boldsymbol{V}^X$ are jointly fed into a Transformer (multi-head self-attention[2] and feed-forward network), which facilitates the integration and aggregation of collaborative information. This process can be expressed as:

$$\boldsymbol{B}^{\text{Colla}} = \text{Transformer}(\boldsymbol{E}^X + \boldsymbol{V}^X) \in \mathbb{R}^{(LT+1)\times D}, \tag{20}$$

where $\boldsymbol{B}^{\text{Colla}}$ represents the output of the collaborative branch.

However, collaborative information is at the item granularity, and the original self-attention mechanism does not align well with this requirement. Therefore, we modify the Transformer's standard attention mechanism to a localized attention restricted to tokens within the same item, enabling more effective aggregation of collaborative signals by the *Item-Level Transformer*. Specifically, this attention operation can be represented as:

$$\text{Attn}(Q, K, V) = \text{softmax}(\frac{QK^\top}{\sqrt{d_k}} + W)V. \tag{21}$$

Here, Q, K, V represent the queries, keys, values, respectively, and $\sqrt{d_k}$ denotes the scaling factor corresponding to the dimension of the keys. $W \in \mathbb{R}^{(LT+1)\times(LT+1)}$ is a specially designed mask introduced to enforce the desired attention

[2]We adopt bidirectional self-attention on the encoder side ($\mathcal{M}(\boldsymbol{E}^X, \boldsymbol{V}^X)$) and unidirectional attention on the decoder side ($\mathcal{M}(\boldsymbol{E}^Y, \boldsymbol{V}^Y)$).

Algorithm 1 Two-stage training procedure of the DiscRec

1: **while** not converged **do** ▷ **Tokenization stage.**
2: Compute $\mathcal{L}_{\mathcal{T}}$ according to Equation (7);
3: Update the parameters of $\mathcal{T}$;
4: **end while**
5: **while** not converged **do** ▷ **Recommendation stage.**
6: Apply tokenization by $\mathcal{T}$ to get X, Y;
7: Compute $\mathcal{M}(\boldsymbol{E}^X, \boldsymbol{V}^X)$ according to Equation (26);
8: Compute $\mathcal{M}(\boldsymbol{E}^Y, \boldsymbol{V}^Y)$ similarly;
9: Compute $\boldsymbol{H}_{\text{Dis}}^{\text{Enc}}$ according to Equation (17);
10: Compute $\boldsymbol{H}_{\text{Dis}}^{\text{Dec}}$ according to Equation (18);
11: Compute $\mathcal{L}_{\mathcal{R}}$ according to Equation (14);
12: Update the parameters of $\mathcal{R}$;
13: **end while**

restriction within tokens of the same item, defined as:

$$W[i][j] = \begin{cases} 0, & \text{if tokens } i \text{ and } j \text{ belong to the same item,} \\ -\infty, & \text{otherwise,} \end{cases} \tag{22}$$

where $W[i][j]$ denotes the value at the i-th row and j-th column of the mask W. Notably, the special tokens [EOS] and [BOS] are considered as belonging to the nearest adjacent item for the purpose of this masking. This localized attention mechanism ensures that the model focuses on intra-item token interactions, thereby enhancing the efficacy of collaborative information aggregation.

Branch Fusion. To preserve the interaction between the two signal types at higher layers of the model, disentanglement is applied exclusively at the embedding level. The outputs of the two branches are then fused based on the *Dynamic Fusion Gate* to produce a unified representation for downstream tasks. Specifically, two gate vectors, $\boldsymbol{G}^{\text{Seman}}$ and $\boldsymbol{G}^{\text{Colla}}$, are learned to adaptively balance and integrate the contributions from each branch. The weights are computed as the inner products between the branch outputs and their corresponding gating vectors, followed by a softmax normalization. This computation process can be formulated as:

$$\boldsymbol{B} = [\boldsymbol{B}^{\text{Seman}}, \boldsymbol{B}^{\text{Colla}}] \in \mathbb{R}^{(LT+1)\times 2\times D}, \tag{23}$$

$$\boldsymbol{G} = [\boldsymbol{G}^{\text{Seman}}, \boldsymbol{G}^{\text{Colla}}] \in \mathbb{R}^{1\times 2\times D}, \tag{24}$$

$$\boldsymbol{S} = \text{softmax}(\boldsymbol{B} \cdot \boldsymbol{G}) \in \mathbb{R}^{(LT+1)\times 2\times 1}. \tag{25}$$

$$\mathcal{M}(\boldsymbol{E}^X, \boldsymbol{V}^X) = \boldsymbol{S}^\top \boldsymbol{B} \in \mathbb{R}^{(LT+1)\times 1\times D}, \tag{26}$$

where $\cdot$ denotes inner product, and $\boldsymbol{S}$ represents the fusion weights of the branch outputs.

B. Training Procedure

Algorithm 1 outlines the overall training process of the proposed DiscRec framework, which consists of two sequential stages: the tokenization stage and the recommendation stage.

In the tokenization stage (lines 1–4), the tokenizer model $\mathcal{T}$ is trained to discretize item representations into semantic IDs. At each iteration, the tokenization loss $\mathcal{L}_{\mathcal{T}}$ is computed (line 2) and used to update the parameters of $\mathcal{T}$ (line 3) until convergence.

In the recommendation stage (lines 5–13), the learned tokenizer $\mathcal{T}$ is first applied to convert historical sequences and target items into tokenized inputs X and Y (line 6). Then, the dual-branch module $\mathcal{M}$ is used to compute disentangled representations for both the encoder and decoder sides (lines 7–8). These representations are used to construct the encoder and decoder hidden states (lines 9–10), which are then used to calculate the recommendation loss $\mathcal{L}_{\mathcal{R}}$ (line 11). Finally, the parameters of the recommender model $\mathcal{R}$ are updated using gradient-based optimization (line 12) until convergence.

C. Complexity Analysis

To further quantify the efficiency of the DiscRec framework, we provide a detailed analysis of its time complexity. Let D denote the model dimension, K the size of each codebook, L the number of codebooks, and T the sequence length.

Tokenization Phase. For a single item, the encoder and decoder layers incur a time complexity of $O(D^2)$. The search operation of each codebook introduces a cost of $O(KD)$, and the computation of the quantization loss adds an additional $O(D)$. Given L codebooks, the intermediate step results in a total complexity of $O(LKD + LD)$. Furthermore, computing the overall reconstruction loss incurs an additional cost of $O(D)$. Therefore, the total complexity of the tokenization phase for a single item is $O(D^2 + LKD)$, and for a sequence of T items, the overall complexity is $O(TD^2 + TLKD)$.

Recommendation Phase. The primary computational cost arises from the self-attention and feed-forward layers, which have a complexity of $O(T^2D + TD^2)$. In addition, the recommendation loss calculation incurs $O(TLKD)$. The collaborative branch contributes $O(T^2D + TD^2)$, while the branch fusion process incurs a cost of $O(TLKD)$.

Therefore, the overall time complexity of DiscRec is $O(TD^2 + T^2D + TLKD)$, which is in the same order as that of the TIGER baseline. Both methods also share similar inference time complexity.

IV. Experiment

In this section, we conduct a series of experiments to answer the following research questions:

RQ1: How does the proposed DiscRec framework perform compared to existing generative recommendation methods?
RQ2: What is the contribution of each component within DiscRec to the overall performance?
RQ3: Does DiscRec effectively achieve the desired item-aware alignment and flexible disentanglement?
RQ4: How well does DiscRec generalize under diverse evaluation settings?
RQ5: Can DiscRec be further improved through alternative architectural designs?

A. Experimental Setting

Table I: Statistical details of the evaluation datasets, where "AvgLen" represents the average length of item sequences.

Datasets	#User	#Item	#Interaction	Sparsity	AvgLen
Beauty	22363	12101	198502	99.93%	8.88
Instruments	24772	9922	206153	99.92%	8.32
Toys	19412	11924	167597	99.93%	8.63
Arts	45141	20956	390832	99.96%	8.66

1) Datasets: We conduct experiments on four subsets of the Amazon Review dataset[3] [23], [24]: Beauty, Musical Instruments, Arts, and Toys. The detailed statistics of these datasets are presented in Table I. Following previous work [25], we apply the 5-core filtering, which removes users and items with fewer than five interactions. To standardize training, each user's interaction history is truncated or padded to a fixed length of 20, keeping the most recent interactions. For data splitting, we adopt the widely used *leave-one-out* strategy. Specifically, for each user, the most recent interaction is held out for testing, the second most recent for validation, and the remaining interactions are used for training.

2) Baselines: The baseline methods used for comparison fall into the following two categories:

a) *Traditional recommendation methods*:

- **MF** [26] decomposes the user-item interactions into the user embeddings and the item embeddings in the latent space.
- **LightGCN** [27] captures high-order user-item interactions through a lightweight graph convolutional network.
- **Caser** [28] leverages both horizontal and vertical convolutional filters to extract sequential patterns from user behavior data, capturing diverse local features.
- **HGN** [29] introduces a hierarchical gating mechanism to adaptively integrate long-term and short-term user preferences derived from historical item sequences.
- **SASRec** [20] employs self-attention mechanisms to capture long-term dependencies in user interaction history.

b) *Generative recommendation methods*:

- **TIGER** [8] adopts the generative retrieval paradigm for sequential recommendation by constructing semantic item IDs derived from text embeddings, thereby enabling semantically meaningful item representations.
- **LETTER** [9] builds upon TIGER by integrating collaborative and diversity regularization into the tokenizer learning process, enhancing the integration of semantic and collaborative signals in the tokenization stage.
- **EAGER** [14] applies a two-stream generative recommender that completely decouples heterogeneous information into separate decoding paths for parallel modeling of semantic and collaborative signals.

3) Evaluation Metrics: To ensure a fair evaluation of sequential recommendation, we adopt two widely used metrics: Recall@K and NDCG@K with $K = 5, 10$. To eliminate sampling bias, we perform full ranking over the entire item set. Additionally, during inference, we apply a constrained decoding strategy [30] that uses a prefix tree to filter out invalid semantic ID prefixes, and set the beam size to 20.

4) Implementation Details: For traditional methods, we follow the implementation from [25]. For generative methods, we adopt a unified model configuration. The recommender is based on the T5 backbone, using 4 Transformer layers with a model dimension of 128, 6 attention heads (dimension 64), a hidden MLP size of 1024, ReLU activations, and a dropout rate of 0.1. The tokenizer employs RQ-VAE for discrete semantic encoding with 4 codebooks, each containing 256 embeddings of dimension 32. We set the weighting coefficient β (in Equation 7) to 0.25. Semantic inputs to RQ-VAE are derived from item titles and descriptions processed by LLaMA2-7B [31]. For collaborative embeddings in LETTER, we follow the original setup and use pretrained SASRec embeddings. For the EAGER baseline, we modify the model dimension to 128 for fair comparison, while keeping other configurations consistent with its official implementation[4].

For our proposed DiscRec, we instantiate it with two different tokenizers by applying it to TIGER and LETTER, denoted as **DiscRec-T** and **DiscRec-L**, respectively. The collaborative branch is configured with 2 attention heads, an attention dimension of 64, an MLP hidden size of 1024 with ReLU activations, and a dropout rate of 0.1. For training the tokenizer model, we train it for 20,000 steps using a batch size of 1,024, the AdamW [32] optimizer, a learning rate of $1e$-3, and a weight decay of $1e$-4. For training the recommender model, we use a batch size of 256 and AdamW optimizer, tuning the learning rate in $5e$-4, $1e$-3 with a fixed weight decay of $1e$-2. For all baseline methods, we follow the hyper-parameter search ranges specified in their original papers.

B. Performance Comparison (RQ1)

We begin by assessing the overall recommendation performance of the compared methods on all four datasets. The summarized results are presented in Table II, yielding the following key observations:

- Applying DiscRec to both TIGER and LETTER consistently improves recommendation performance (DiscRec-T>TIGER, DiscRec-L>LETTER), with DiscRec-L achieving the best overall results. These gains stem from the incorporation of item-awareness within the framework, which aligns the Transformer-based recommender architecture more closely with the recommendation task, and from explicit signal disentanglement that facilitates more effective and coordinated learning of semantic and collaborative information.
- Traditional recommendation methods generally underperform compared to generative approaches, primarily due to

[3]https://nijianmo.github.io/amazon/index.html

[4]EAGER's original data processing does not correctly sort user interaction histories by timestamp, potentially resulting in overestimated performance. In our experiments, we address this by applying proper chronological ordering. Further details are available at https://github.com/yewzz/EAGER/issues/5

Table II: Overall performance comparison between baselines and our proposed DiscRec. Specifically, DiscRec is applied on top of TIGER and LETTER–referred to as DiscRec-T and DiscRec-L, respectively–with "RI" denoting the relative improvement compared to TIGER and LETTER. The best and second-best results are highlighted in bold and underlined, respectively.

Dataset	Metric	Traditional Method					Generative Method						
		MF	LightGCN	Caser	HGN	SASRec	EAGER	TIGER	DiscRec-T	RI (%)	LETTER	DiscRec-L	RI (%)
Beauty	Recall@5	0.0202	0.0228	0.0279	0.0344	0.0403	0.0367	0.0384	0.0414	7.8	0.0424	**0.0472**	11.3
	Recall@10	0.0379	0.0421	0.0456	0.0564	0.0589	0.0534	0.0609	0.0656	7.7	0.0612	**0.0704**	15.0
	NDCG@5	0.0122	0.0136	0.0172	0.0214	0.0271	0.0266	0.0256	0.0270	5.5	0.0264	**0.0308**	16.7
	NDCG@10	0.0178	0.0198	0.0229	0.0284	0.0331	0.0322	0.0329	0.0348	5.8	0.0334	**0.0382**	14.4
Instruments	Recall@5	0.0738	0.0757	0.0770	0.0854	0.0857	0.0599	0.0865	0.0901	4.2	0.0872	**0.0932**	6.9
	Recall@10	0.0967	0.1010	0.0995	0.1086	0.1083	0.0745	0.1062	0.1101	3.7	0.1082	**0.1153**	6.6
	NDCG@5	0.0473	0.0472	0.0639	0.0723	0.0715	0.0506	0.0736	0.0752	2.2	0.0747	**0.0791**	5.9
	NDCG@10	0.0547	0.0554	0.0711	0.0798	0.0788	0.0553	0.0799	0.0817	2.3	0.0814	**0.0862**	5.9
Toys	Recall@5	0.0218	0.0253	0.0243	0.0328	0.0357	0.0301	0.0316	0.0367	16.1	0.0334	**0.0403**	20.7
	Recall@10	0.0350	0.0413	0.0404	0.0417	0.0602	0.0451	0.0511	0.0583	14.1	0.0542	**0.0634**	17.0
	NDCG@5	0.0123	0.0149	0.0150	0.0243	0.0228	0.0215	0.0204	0.0235	15.2	0.0225	**0.0254**	12.9
	NDCG@10	0.0165	0.0200	0.0201	0.0272	0.0306	0.0268	0.0267	0.0304	13.9	0.0292	**0.0328**	12.3
Arts	Recall@5	0.0473	0.0464	0.0571	0.0667	0.0758	0.0555	0.0807	0.0822	1.9	0.0841	**0.0850**	1.1
	Recall@10	0.0753	0.0755	0.0781	0.0910	0.0945	0.0694	0.1017	0.1061	4.3	0.1081	**0.1110**	2.7
	NDCG@5	0.0271	0.0244	0.0407	0.0516	0.0632	0.0472	0.0640	0.0659	3.0	0.0675	**0.0684**	1.3
	NDCG@10	0.0361	0.0338	0.0474	0.0595	0.0693	0.0518	0.0707	0.0736	4.1	0.0752	**0.0768**	2.1

their reliance on ID embeddings, which limits their ability to capture hierarchical semantic structures at multiple levels of granularity. In contrast, generative models enhanced with RQ-VAE are capable of modeling such structures, allowing for more fine-grained distinctions between semantically similar items through richer, continuous representations.

- LETTER outperforms TIGER, largely due to the additional supervision introduced by incorporating collaborative signals during the tokenization stage. Specifically, LETTER constructs an auxiliary loss based on collaborative embeddings, effectively bridging the gap between items with similar semantics but divergent interaction patterns.
- Although EAGER adopts a two-stream generative design for disentanglement, its performance is limited; on the Instruments dataset, it is even outperformed by traditional baselines like MF. This suggests that processing collaborative and semantic signals independently may lead to over-separation, hindering their integration and diminishing the model's ability to capture their complementary strengths.

C. Ablation Study (RQ2)

To validate the design rationale behind DiscRec, we perform a thorough evaluation by systematically ablation of each key component, yielding a set of model variants. Specifically, starting from DiscRec-T, we introduce the following variants for comparison:

- **w/o IPE**: This variant removes the item-level position embedding from the collaborative branch, relying solely on the token embeddings to model collaborative signals.
- **w/o TF**: This variant eliminates the whole introduced Transformer equipped with localized attention operation.
- **w/o Gating**: This variant eliminates the adaptive gating fusion mechanism, replacing it with a direct summation of the two branch representations.

In addition to the above basic variants, we further design several alternative variants based on TIGER to explore potential strategies for incorporating item-awareness:

- **w/ PE**: This variant adopts standard learnable positional embeddings, implemented as an embedding table with a size equal to the maximum length of the input token sequence.
- **w/ IE**: This variant introduces an additional item ID embedding, which is concatenated to the end of each item's original token embedding. The size of the embedding table corresponds to the total number of unique items.
- **w/ IPE**: This variant directly applies the item-level positional embedding in DiscRec by summation without any further modifications.

Lastly, to rule out the impact of increased model capacity, we introduce an additional variant:

- **w/ 5-Layer**: The number of transformer layers in TIGER is increased to 5, resulting in a parameter increase greater than that introduced by DiscRec to ensure a fair comparison.

Table III illustrates the comparison results on Beauty, from which we draw the following observations:

- The removal of item-level positional embedding, Transformer with localized attention and adaptive gating all leads to noticeable performance degradation, confirming the effectiveness of these specific design choices. Among them, w/o IPE and w/o Attn results in a more substantial decline in performance. Since both components are applied within the collaborative branch, this finding underscores the critical importance of effectively leveraging collaborative signals.

Table III: Ablation study of DiscRec-T on Beauty.

Method	Recall@5	NDCG@5	Recall@10	NDCG@10
DiscRec-T	**0.0414**	**0.0270**	**0.0656**	**0.0348**
w/o IPE	0.0382	0.0246	0.0619	0.0322
w/o TF	0.0377	0.0253	0.0605	0.0327
w/o Gating	0.0403	0.0262	0.0629	0.0334
TIGER	0.0381	0.0243	0.0593	0.0311
w/ PE	0.0369	0.0237	0.0597	0.0310
w/ IE	0.0396	0.0257	0.0619	0.0329
w/ IPE	0.0372	0.0249	0.0600	0.0322
w/ 5-Layer	0.0367	0.0240	0.0590	0.0312

- Among the three TIGER variants incorporating item-awareness, both w/ IPE and w/ IE lead to consistent performance improvements, whereas w/ PE results in negligible change. This suggests that the former two approaches effectively introduce item-level structural patterns, thereby improving alignment with the recommendation task. Although w/ IE yields greater performance gains, its reliance on a large item embedding table imposes scalability constraints. In contrast, the proposed IPE offers a more scalable and cost-effective solution, thereby enhancing practicality in large-scale recommendation scenarios.
- Simply increasing the recommender model capacity in TIGER does not lead to significant performance gains, likely due to its misaligned architecture and the entanglement of information. This underscores that the improvements achieved by our method stem not merely from increased parameters, but from enhanced task alignment and effective disentanglement of semantic and collaborative signals.

D. Effectiveness Analysis (RQ3)

We propose DiscRec to address two fundamental challenges: token-item misalignment, and semantic–collaborative signal entanglement. Therefore, we next evaluate the extent to which these challenges have been successfully mitigated.

1) Item-Aware Alignment: First, to validate the effectiveness of DiscRec in achieving item-aware alignment, we visualize the attention distributions across the four encoder layers of the recommender model. Using the Beauty test set, we average attention scores over six heads per layer and present the results as heatmaps. Since the average user interaction length in the test set of Beauty is 7.19–shorter than the maximum sequence length of 20–many padding tokens produce extensive low-activation regions in the heatmaps, complicating interpretation. To enhance clarity, we crop all heatmaps to a fixed size of 28×28 based on the average interaction length. For comparison, we include the TIGER and LETTER baseline, which excludes all DiscRec-specific components.

As shown in Figure 4 and Figure 5, the attention heatmaps across all four encoder layers of DiscRec-T and DiscRec-L display a clear 4×4 grid-like segmentation pattern. This regularity results from discretizing each item into four tokens,

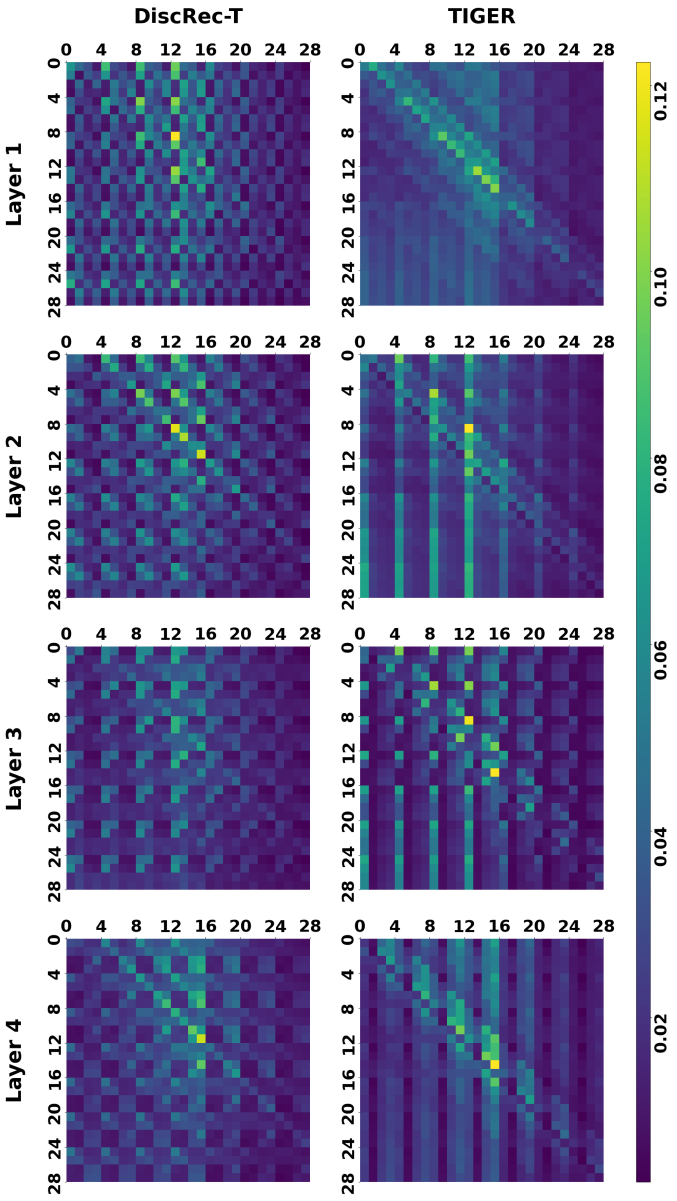

Fig. 4: Layer-wise attention heatmaps for the four Transformer layers of DiscRec-T and TIGER, cropped to 28×28 to minimize the impact of padding.

which induces structured attention distributions aligned with item-level granularity. In contrast, such patterns are not evident in the corresponding layers of the TIGER and LETTER baselines. These observations suggest that DiscRec effectively models fine-grained item-level structure throughout the encoding process, thereby improving task alignment and enhancing the extraction of collaborative signals.

2) Semantic-Collaborative Signal Disentanglement with Flexible Fusion: To assess the effectiveness of DiscRec in achieving signal disentanglement and flexible fusion, we next examine the distribution of semantic and collaborative signals. Specifically, we focus on the distribution of three types of

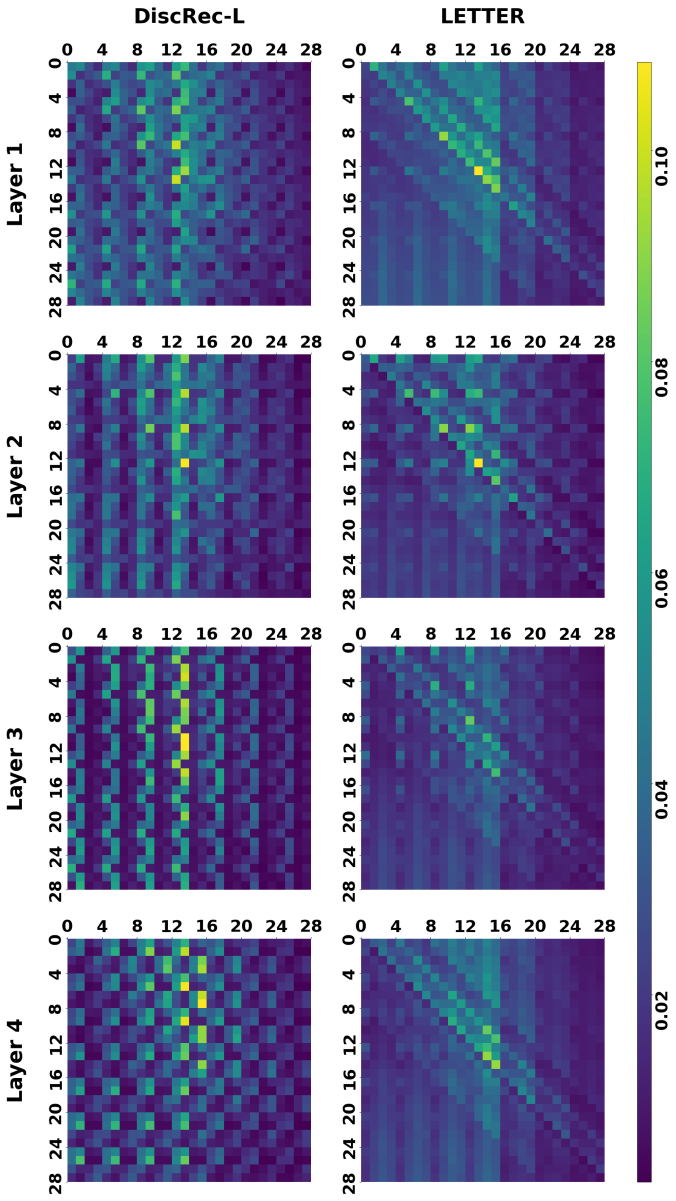

Fig. 5: Layer-wise attention heatmaps for the four Transformer layers of DiscRec-L and LETTER, cropped to 28×28 to minimize the impact of padding.

embeddings in relation to the semantic ID index: code embeddings produced by the tokenizer model (Code Emb.), semantic token embeddings from the semantic branch (Seman. Emb.), and collaborative token embeddings from the collaborative branch (Colla. Emb.). Using the Beauty dataset, we compute the average embedding norm as a measure of information capacity and perform this analysis for both DiscRec-T and DiscRec-L.

As illustrated in Figure 6, two key observations emerge:

- For both DiscRec-T and DiscRec-L, the semantic token embeddings exhibit distribution patterns similar to those of the code embeddings, showing a consistent decline across token indices—a characteristic inherent to residual quantization. This indicates that the semantic information originally encoded in the semantic ID sequences has been successfully disentangled and captured by the semantic branch.
- In contrast, the collaborative token embeddings exhibit a markedly different distribution pattern, increasing with indices of the semantic ID. This suggests that **during item decoding, the model gradually shifts its focus from semantic signals toward collaborative signals**. This observation aligns with our hypothesis in Figure 1 that **semantic and collaborative signals display significantly distinct distribution patterns in generative recommendation**. Moreover, these findings confirm that DiscRec could effectively disentangle two types of signals.

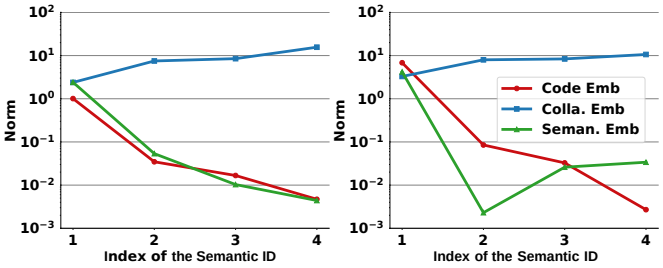

Fig. 6: Embedding norm distribution comparison of code embeddings from the tokenizer, semantic token embeddings, and collaborative token embeddings from the recommender.

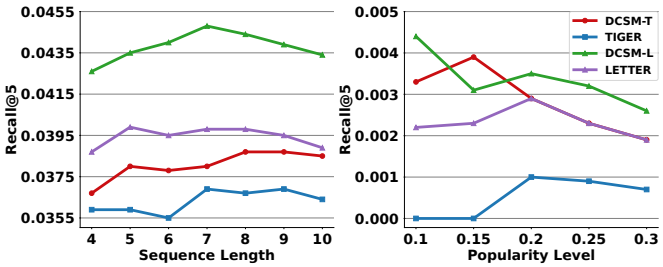

Fig. 7: Performance comparison on different sequence lengths and popularity levels on Beauty.

E. Generalization Analysis (RQ4)

To evaluate DiscRec's generalization, we test it on the Beauty dataset across three scenarios: varying input sequence lengths, different item popularity levels, and alternative data splits:

- **Sequence Length**: To evaluate robustness across limited user history lengths, we group test samples by input sequence lengths from 4 to 10, noting that the minimum length of 4 results from our 5-core preprocessing rules, and assess model performance within each group.
- **Popularity Level**: To assess robustness across varying item popularity levels, we group test samples according to the popularity of their target items. We then evaluate on subsets where the target items fall within the lowest 10%, 15%, 20%, 25%, and 30% of the popularity distribution.

Table IV: Performance comparison under user-level random split on Beauty.

Method	Recall@5	NDCG@5	Recall@10	NDCG@10
DiscRec-T	0.0786	0.0530	0.1157	0.0649
TIGER	0.0754	0.0509	0.1112	0.0625
DiscRec-L	0.0847	0.0581	0.1208	0.0697
LETTER	0.0796	0.0544	0.1165	0.0662

- **Data Split**: To evaluate robustness under different data partitioning schemes, we replace the original leave-one-out split with a user-level random split. Specifically, the dataset is shuffled at the user level and divided into training, validation, and test sets in an 8:1:1 ratio. All models are then retrained from scratch using this new split.

The results are shown in Figure 7 and Table IV, from which we make the following observations:

- Across varying sequence lengths and item popularity levels, DiscRec consistently achieves performance improvements over both TIGER and LETTER. This can be attributed to its more balanced integration of collaborative and semantic signals, enabling robust gains across diverse scenarios.
- Under the user-based split, DiscRec continues to deliver consistent performance improvements, suggesting that its effectiveness is not contingent on specific user behavior patterns. This highlights the model's robustness and strong generalization capability across diverse user distributions.

F. Potential Design Analysis (RQ5)

To further investigate the potential extensions of DiscRec's architectural components, we conduct exploratory experiments with four enhanced variants of the framework:

- **AllLayer**: This variant extends the application of DiscRec beyond the embedding layer to all Transformer layers.
- **OneQuery**: Inspired by Q-former designs, this variant replaces the localized attention in the collaborative branch of DiscRec with an attention mechanism that uses a single learnable query embedding for all tokens.
- **TokenAvg**: This variant introduces an additional step to the Transformer output in the collaborative branch, where token representations corresponding to the same item are averaged to form a unified item-level representation.
- **SelfGating**: This variant modifies DiscRec's adaptive gating mechanism by adopting a self-gating strategy inspired by HGN [29]. Specifically, it computes gating vectors via an MLP with sigmoid activation applied to the outputs of the two branches, and uses element-wise multiplication to modulate the original representations.

The performance of these four variants is presented in Table V, and we make the following observations:

- Applying DiscRec to all Transformer layers (AllLayer) results in a significant performance decline, falling below the TIGER baseline, which suggests that decoupling is most suitable at the embedding layer. This is likely because the higher layers primarily facilitate information interaction, where decoupling may be detrimental.
- OneQuery performs slightly worse than TIGER, suggesting that learning a unified collaborative query embedding from sequential patterns is challenging. This highlights the advantage of directly using self-attention for modeling collaborative signals. TokenAvg also shows slightly inferior performance compared to DiscRec-T, indicating that preserving token-level personalized collaborative information is more effective than simple averaging-based aggregation.
- Self-Gating shows no significant performance difference compared to DiscRec-T, suggesting that directly learning a gating vector is a simple yet effective approach for adaptively fusing the two types of information.

Table V: Exploratory experimental results on Beauty.

Method	Recall@5	NDCG@5	Recall@10	NDCG@10
DiscRec-T	**0.0414**	**0.0270**	**0.0656**	**0.0348**
TIGER	0.0381	0.0243	0.0593	0.0311
AllLayer	0.0313	0.0191	0.0504	0.0252
OneQuery	0.0351	0.0225	0.0563	0.0293
TokenAvg	0.0397	0.0255	0.0614	0.0324
SelfGating	0.0406	0.0268	0.0640	0.0343

V. Related Work

In this section, we navigate through existing research on sequential recommendation, generative recommendation and collaborative information modeling.

A. Sequential Recommendation

Sequential recommendation aims to predict the next item a user will interact with based on their historical behavior. Early approaches primarily utilized Markov Chains to capture item transition dynamics [33], [34]. Later developments introduced a range of deep learning architectures. Specifically, GRU4Rec [35] pioneered the use of recurrent neural networks (RNNs) in session-based recommendation, whereas Caser [28] adopted convolutional neural networks (CNNs) to model user behavior. More recently, methods like SASRec [20] and BERT4Rec [36] have employed self-attention mechanisms, achieving state-of-the-art results in sequential recommendation tasks. To enrich item representations with contextual signals, FDSA [37] incorporates a self-attention module that effectively captures item attribute information. In a similar vein, S^3-Rec [25] enhances performance by maximizing mutual information across multiple types of contextual data. Although these methods have significantly advanced the field, they remain fundamentally discriminative in nature and thus face inherent limitations in generalization ability.

B. Generative Recommendation

Motivated by the remarkable success of generative AI, there has been a growing interest in transitioning recommenda-

tion paradigms from discriminative to generative frameworks across both academia and industry. This emerging generative recommendation paradigm typically consists of two core stages: a tokenization stage, which encodes item semantics into discrete representations, and a generation stage, which produces item predictions based on these representations. TIGER [8] is among the pioneering efforts in this direction, leveraging RQ-VAE to discretize item representations into semantic tokens and employing an encoder–decoder architecture to model user behavior sequences for item generation. Subsequent research has aimed to enhance these two stages. For the tokenization stage, several works have explored the additional integration of auxiliary information to improve the adaptability of semantic IDs [9], [12], [30], [38]–[40]. For example, LETTER [9] incorporates collaborative and diversity regularization into the token learning process, while ActionPiece [38] introduces context-awareness, enabling tokens to reflect different semantics depending on their surrounding context. On the recommendation side, researchers have proposed diverse architectural designs [3], [11], [14], [41] to better capture user–item interactions. For instance, EAGER [14] presents a two-stream architecture that parallelly models semantic and collaborative signals. LC-Rec [41], on the other hand, explores the integration of large language models (LLMs) to more effectively leverage semantic IDs.

C. Collaborative Information Modeling

In traditional ID-based recommendation paradigms, the modeling of collaborative signals derived from co-occurrence patterns in user-item interactions serves as a cornerstone. Recently, with the rapid adoption of large language models (LLMs), numerous studies have explored leveraging the world knowledge and reasoning capabilities of LLMs to perform recommendation tasks. For example, TALLRec [42] fine-tunes LLMs on recommendation data to perform click-through rate (CTR) prediction tasks. BIGRec [43] first grounds LLMs in the recommendation space by fine-tuning them to generate meaningful tokens corresponding to items, and then retrieves actual items that align with the generated tokens. However, these methods typically represent users and items as text tokens, relying primarily on textual semantics, which inherently fail to capture collaborative signals. As a result, a growing body of research has emerged to incorporate collaborative information into LLM-based recommendation systems. For instance, CoLLM [44] integrates collaborative signals by leveraging an external traditional recommendation model and mapping the resulting representations into the input embedding space of the LLM, thereby forming collaborative-aware embeddings for downstream tasks. Similarly, LLaRA [45] incorporates ID embeddings and historical interaction sequences directly into the prompt to enhance the model's capacity for sequential recommendation.

In the aforementioned two-stage generative recommendation paradigm, efforts have also been made to leverage both collaborative and semantic signals. These approaches can be broadly categorized into two groups. The first group enhances the tokenizer with collaborative supervision signals. For instance, LETTER [9] incorporates embeddings from a pre-trained ID-based collaborative filtering model as additional supervision, while ETEGRec [10] utilizes outputs from the downstream recommender model for the same purpose. The second group adopts dual-branch architectures, where semantic and collaborative signals are modeled independently. For example, EAGER [14] employs separate decoders for the modeling of semantic and collaborative tasks. SC-Rec [17] leverages a unified LLM as a reasoning engine, performing independent task learning on both collaborative and semantic token sequences. However, these methods fail to address the challenges of token-item misalignment and semantic–collaborative signal entanglement. that we aim to resolve in this work. In contrast, our work addresses this gap through a more in-depth exploration of this disentanglement problem in generative recommendation.

VI. Conclusion

This paper proposed DiscRec, a novel framework for generative recommendation that addresses two fundamental challenges: token-item misalignment, and semantic–collaborative signal entanglement. Traditional token-level architectures fail to capture item-level granularity and entangle heterogeneous signals in a shared embedding space, which hinders effective learning. To tackle these issues, DiscRec introduces item-level position embeddings to capture item-aware structural information within token sequences. In addition, it employs a dual-branch architecture to disentangle semantic and collaborative signals at the embedding layer, with a gating mechanism that adaptively fuses the two representations, enabling flexible disentanglement while retaining cross-signal interactions at deeper layer. Overall, DiscRec offers a lightweight solution for achieving item-aware alignment, and flexible signal disentanglement and fusion.

In future work, we will explore several promising directions to further enhance the DiscRec framework. First, we plan to adapt the DiscRec framework to LLM-based recommendation scenarios. Additionally, we plan to investigate more advanced disentanglement strategies, such as contrastive learning or mutual information minimization, to more effectively separate semantic and collaborative signals.

VII. AI-Generated Content Acknowledgement

We leveraged ChatGPT (OpenAI, GPT-4-turbo, 2025) exclusively to polish the prose of the manuscript drafted by the authors, focusing on grammar correction, clarity enhancement, and overall readability improvement. During development, it was also employed as a debugging aid for code. At no stage did ChatGPT contribute to the generation of substantive research content—this includes formulation of concepts or algorithms, implementation of code, data analysis, presentation of experimental findings, or creation of tables and figures.

REFERENCES

[1] W. Wang, X. Lin, F. Feng, X. He, and T.-S. Chua, "Generative recommendation: Towards next-generation recommender paradigm," *arXiv preprint arXiv:2304.03516*, 2023.

[2] J. Deng, S. Wang, K. Cai, L. Ren, Q. Hu, W. Ding, Q. Luo, and G. Zhou, "Onerec: Unifying retrieve and rank with generative recommender and iterative preference alignment," *arXiv preprint arXiv:2502.18965*, 2025.

[3] J. Zhai, L. Liao, X. Liu, Y. Wang, R. Li, X. Cao, L. Gao, Z. Gong, F. Gu, J. He, Y. Lu, and Y. Shi, "Actions speak louder than words: trillion-parameter sequential transducers for generative recommendations," in *Proceedings of the 41st International Conference on Machine Learning*, ser. ICML'24. JMLR.org, 2024.

[4] A. Badrinath, P. Agarwal, L. Bhasin, J. Yang, J. Xu, and C. Rosenberg, "Pinrec: Outcome-conditioned, multi-token generative retrieval for industry-scale recommendation systems," *arXiv preprint arXiv:2504.10507*, 2025.

[5] R. Han, B. Yin, S. Chen, H. Jiang, F. Jiang, X. Li, C. Ma, M. Huang, X. Li, C. Jing *et al.*, "Mtgr: Industrial-scale generative recommendation framework in meituan," *arXiv preprint arXiv:2505.18654*, 2025.

[6] L. Zhang, K. Song, Y. Q. Lee, W. Guo, H. Wang, Y. Li, H. Guo, Y. Liu, D. Lian, and E. Chen, "Killing two birds with one stone: Unifying retrieval and ranking with a single generative recommendation model," *arXiv preprint arXiv:2504.16454*, 2025.

[7] D. Lee, C. Kim, S. Kim, M. Cho, and W.-S. Han, "Autoregressive image generation using residual quantization," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022, pp. 11 523–11 532.

[8] S. Rajput, N. Mehta, A. Singh, R. Keshavan, T. Vu, L. Heidt, L. Hong, Y. Tay, V. Q. Tran, J. Samost, M. Kula, E. H. Chi, and M. Sathiamoorthy, "Recommender systems with generative retrieval," in *Proceedings of the 37th International Conference on Neural Information Processing Systems*, ser. NIPS '23. Red Hook, NY, USA: Curran Associates Inc., 2023, pp. 10 299–10 315.

[9] W. Wang, H. Bao, X. Lin, J. Zhang, Y. Li, F. Feng, S.-K. Ng, and T.-S. Chua, "Learnable item tokenization for generative recommendation," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, ser. CIKM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2400–2409.

[10] E. Liu, B. Zheng, C. Ling, L. Hu, H. Li, and W. X. Zhao, "Generative recommender with end-to-end learnable item tokenization," in *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '25. New York, NY, USA: Association for Computing Machinery, 2025.

[11] Z. Si, Z. Sun, J. Chen, G. Chen, X. Zang, K. Zheng, Y. Song, X. Zhang, J. Xu, and K. Gai, "Generative retrieval with semantic tree-structured identifiers and contrastive learning," in *Proceedings of the 2024 Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, ser. SIGIR-AP 2024. New York, NY, USA: Association for Computing Machinery, 2024, p. 154–163.

[12] H. Qu, W. Fan, Z. Zhao, and Q. Li, "Tokenrec: learning to tokenize id for llm-based generative recommendation," *arXiv preprint arXiv:2406.10450*, 2024.

[13] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.

[14] Y. Wang, J. Xun, M. Hong, J. Zhu, T. Jin, W. Lin, H. Li, L. Li, Y. Xia, Z. Zhao, and Z. Dong, "Eager: Two-stream generative recommender with behavior-semantic collaboration," in *Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, ser. KDD '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 3245–3254.

[15] Y. Wang, Z. Ren, W. Sun, J. Yang, Z. Liang, X. Chen, R. Xie, S. Yan, X. Zhang, P. Ren, Z. Chen, and X. Xin, "Content-based collaborative generation for recommender systems," in *Proceedings of the 33rd ACM International Conference on Information and Knowledge Management*, ser. CIKM '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 2420–2430.

[16] L. Xiao, H. Wang, C. Wang, L. Ji, Y. Wang, J. Zhu, Z. Dong, R. Zhang, and R. Li, "Progressive collaborative and semantic knowledge fusion for generative recommendation," *arXiv preprint arXiv:2502.06269*, 2025.

[17] T. Kim, S. Yoon, S. Kang, J. Yeo, and D. Lee, "Sc-rec: Enhancing generative retrieval with self-consistent reranking for sequential recommendation," *arXiv preprint arXiv:2408.08686*, 2024.

[18] M. Hong, Y. Xia, Z. Wang, J. Zhu, Y. Wang, S. Cai, X. Yang, Q. Dai, Z. Dong, Z. Zhang, and Z. Zhao, "Eager-llm: Enhancing large language models as recommenders through exogenous behavior-semantic integration," in *Proceedings of the ACM on Web Conference 2025*, ser. WWW '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 2754–2762.

[19] M. Cheng, H. Zhang, Q. Liu, F. Yuan, Z. Li, Z. Huang, E. Chen, J. Zhou, and L. Li, "Empowering sequential recommendation from collaborative signals and semantic relatedness," in *International Conference on Database Systems for Advanced Applications*. Springer, 2024, pp. 196–211.

[20] W.-C. Kang and J. McAuley, "Self-attentive sequential recommendation," in *2018 IEEE international conference on data mining (ICDM)*, IEEE. IEEE Computer Society, 2018, pp. 197–206.

[21] M. Oyama, S. Yokoi, and H. Shimodaira, "Norm of word embedding encodes information gain," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 2108–2130.

[22] H. Kurita, G. Kobayashi, S. Yokoi, and K. Inui, "Contrastive learning-based sentence encoders implicitly weight informative words," in *Findings of the Association for Computational Linguistics: EMNLP 2023*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 10 932–10 947.

[23] R. He and J. McAuley, "Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering," in *Proceedings of the 25th International Conference on World Wide Web*, ser. WWW '16. Republic and Canton of Geneva, CHE: International World Wide Web Conferences Steering Committee, 2016, p. 507–517.

[24] J. Ni, J. Li, and J. McAuley, "Justifying recommendations using distantly-labeled reviews and fine-grained aspects," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 188–197.

[25] K. Zhou, H. Wang, W. X. Zhao, Y. Zhu, S. Wang, F. Zhang, Z. Wang, and J.-R. Wen, "S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization," in *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, ser. CIKM '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 1893–1902.

[26] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[27] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang, and M. Wang, "Lightgcn: Simplifying and powering graph convolution network for recommendation," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 639–648.

[28] J. Tang and K. Wang, "Personalized top-n sequential recommendation via convolutional sequence embedding," in *Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining*, ser. WSDM '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 565–573.

[29] C. Ma, P. Kang, and X. Liu, "Hierarchical gating networks for sequential recommendation," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, ser. KDD '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 825–833.

[30] W. Hua, S. Xu, Y. Ge, and Y. Zhang, "How to index item ids for recommendation foundation models," in *Proceedings of the Annual International ACM SIGIR Conference on Research and Development in Information Retrieval in the Asia Pacific Region*, ser. SIGIR-AP '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 195–204.

[31] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.

[32] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.

[33] R. He and J. McAuley, "Fusing similarity models with markov chains for sparse sequential recommendation," in *2016 IEEE 16th International Conference on Data Mining (ICDM)*, 2016, pp. 191–200.

[34] S. Rendle, C. Freudenthaler, and L. Schmidt-Thieme, "Factorizing personalized markov chains for next-basket recommendation," in *Proceedings of the 19th International Conference on World Wide Web*, ser. WWW '10. New York, NY, USA: Association for Computing Machinery, 2010, p. 811–820.

[35] B. Hidasi and A. Karatzoglou, "Recurrent neural networks with top-k gains for session-based recommendations," in *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*, ser. CIKM '18. New York, NY, USA: Association for Computing Machinery, 2018, p. 843–852.

[36] F. Sun, J. Liu, J. Wu, C. Pei, X. Lin, W. Ou, and P. Jiang, "Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer," in *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, ser. CIKM '19. New York, NY, USA: Association for Computing Machinery, 2019, p. 1441–1450.

[37] T. Zhang, P. Zhao, Y. Liu, V. S. Sheng, J. Xu, D. Wang, G. Liu, and X. Zhou, "Feature-level deeper self-attention network for sequential recommendation," in *Proceedings of the 28th International Joint Conference on Artificial Intelligence*, ser. IJCAI'19. AAAI Press, 2019, p. 4320–4326.

[38] Y. Hou, J. Ni, Z. He, N. Sachdeva, W.-C. Kang, E. H. Chi, J. McAuley, and D. Z. Cheng, "Actionpiece: Contextually tokenizing action sequences for generative recommendation," *arXiv preprint arXiv:2502.13581*, 2025.

[39] S. Geng, S. Liu, Z. Fu, Y. Ge, and Y. Zhang, "Recommendation as language processing (rlp): A unified pretrain, personalized prompt & predict paradigm (p5)," in *Proceedings of the 16th ACM Conference on Recommender Systems*, ser. RecSys '22. New York, NY, USA: Association for Computing Machinery, 2022, p. 299–315.

[40] J. Tan, S. Xu, W. Hua, Y. Ge, Z. Li, and Y. Zhang, "Idgenrec: Llm-recsys alignment with textual id learning," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 355–364.

[41] B. Zheng, Y. Hou, H. Lu, Y. Chen, W. X. Zhao, M. Chen, and J.-R. Wen, "Adapting large language models by integrating collaborative semantics for recommendation," in *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. IEEE, 2024, pp. 1435–1448.

[42] K. Bao, J. Zhang, Y. Zhang, W. Wang, F. Feng, and X. He, "Tallrec: An effective and efficient tuning framework to align large language model with recommendation," in *Proceedings of the 17th ACM Conference on Recommender Systems*, ser. RecSys '23. New York, NY, USA: Association for Computing Machinery, 2023, p. 1007–1014.

[43] K. Bao, J. Zhang, W. Wang, Y. Zhang, Z. Yang, Y. Luo, C. Chen, F. Feng, and Q. Tian, "A bi-step grounding paradigm for large language models in recommendation systems," *ACM Trans. Recomm. Syst.*, vol. 3, no. 4, Apr. 2025.

[44] Y. Zhang, F. Feng, J. Zhang, K. Bao, Q. Wang, and X. He, "Collm: Integrating collaborative embeddings into large language models for recommendation," *IEEE Transactions on Knowledge and Data Engineering*, vol. 37, no. 5, pp. 2329–2340, 2025.

[45] J. Liao, S. Li, Z. Yang, J. Wu, Y. Yuan, X. Wang, and X. He, "Llara: Large language-recommendation assistant," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 1785–1795.